CPA security, pseudorandom functions, and block ciphers



CS 161 Spring 2026 - Lecture 7

Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad – CPA security – Counter-mode encryption – CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)

Building blocks	– Pseudorandom functions – Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem

Recap: Secret-key encryption

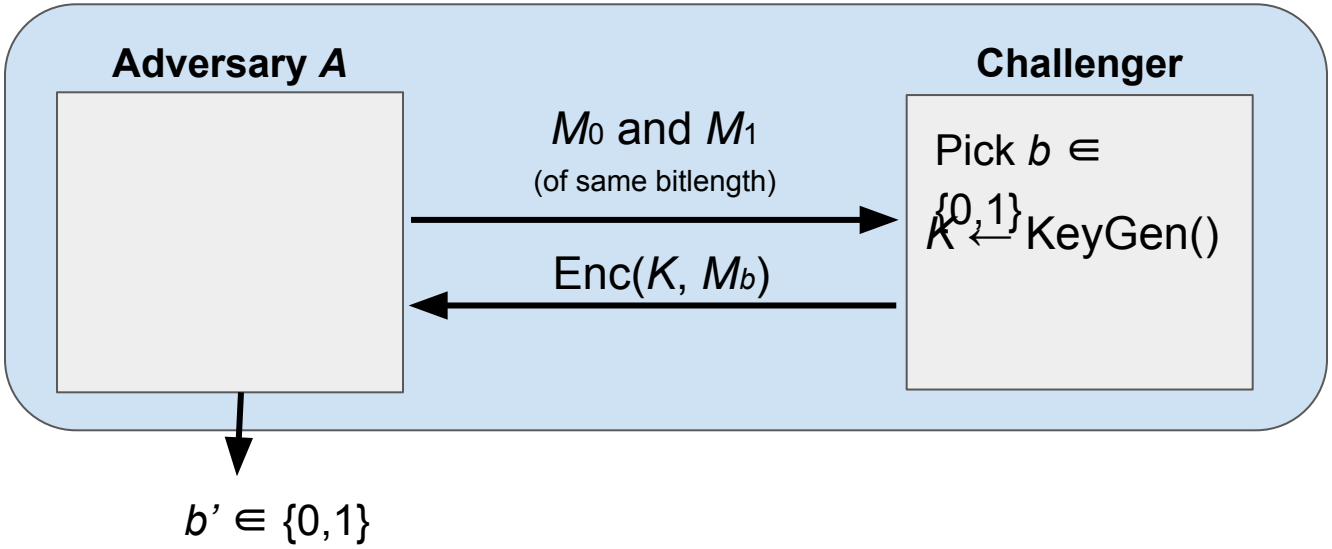
Three efficient algorithms:

- KeyGen() → K
- Enc(K, M) → C
- Dec(K, C) → M

To be useful, these algorithms must also have:

- Correctness: For all messages in msg space, for all $K \leftarrow$ KeyGen():
  Dec(K, Enc(K, M)) = M.
- Security: Semantic security for one-time key *(next slide)*
  - A weak starting-point security goal. We will do better today!

Recap: Semantic security for one-time key



An encryption scheme has *semantic security for a one-time key* if, for all "efficient" adversaries A, $|\Pr[A \text{ outputs } b] - \frac{1}{2}| \leq$ "negligible."

Recap: One-time pad

Your best choice for a desert-island cryptosystem!

- KeyGen() → Output random n-bit string
- Enc(K, M) → $K \oplus M$
- Dec(K, C) → $K \oplus C$

Pros:
– Perfectly secure, in the sense of semantic security for one-time key
– Simple

Cons:
– Key is as long as message. To encrypt a movie, you need GBs of keys
– Key is one-time. Encrypt twice with same key ⇒

Question: What is an example of a cipher that is NOT semantically secure for a one-time key?

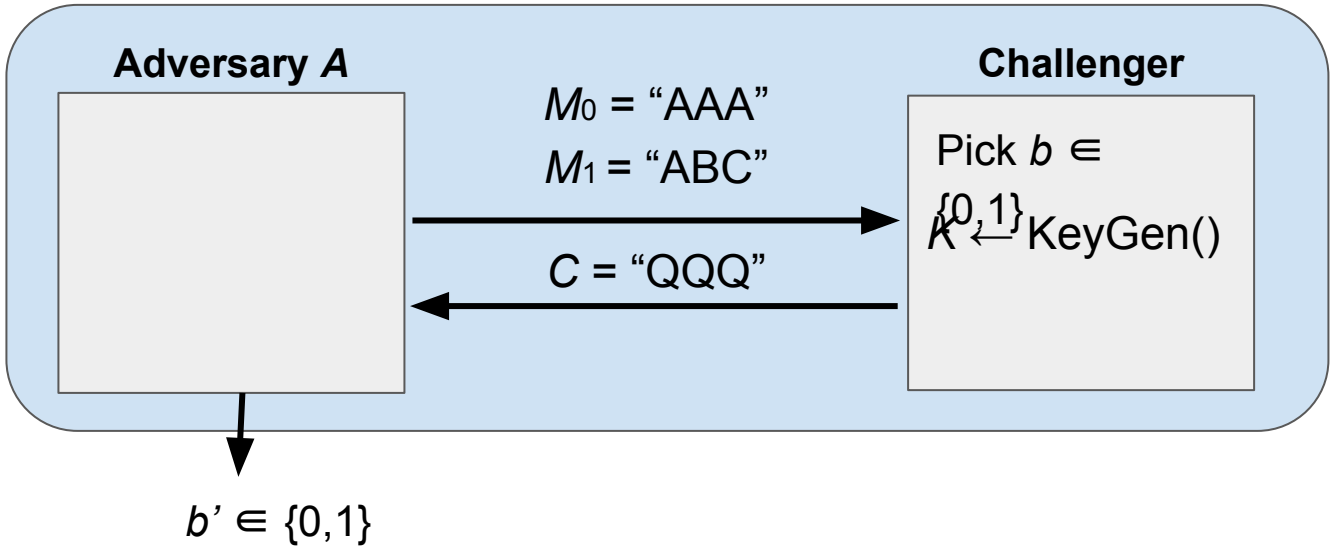
Please also ask during lecture! Use Zoom chat if you prefer.

Caesar cipher - Used by Julius Caesar over 2,000 years ago

- KeyGen():
  - Output $K \leftarrow_R \{0, \ldots, 25\}$
- Enc(K, M):
  - Replace each letter in M with the letter K positions **later** in the alphabet
  - If $K = 3$, it encrypts "AAA" → "CCC" and "DOG" → "GRJ"
- Dec(K, C):
  - Replace each letter in C with the letter K positions **earlier** in the alphabet
  - If $K = 3$, it decrypts "CCC" → "AAA" and "GRJ" → "DOG"

Caesar cipher is not semantically secure for one-time key

Problem: Caesar cipher preserves letter frequencies

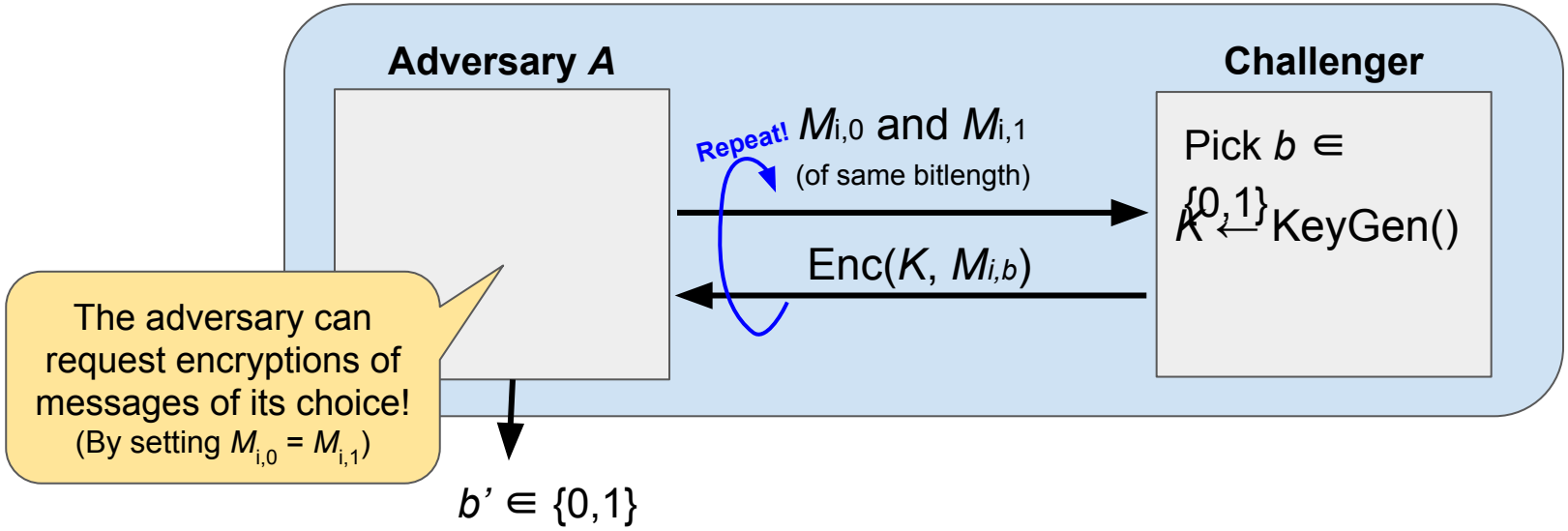


A stronger security definition
for secret-key encryption:

CPA security



Semantic security for many-time key ("CPA security", "IND-CPA security")



An encryption scheme is *CPA secure* if,
for all "efficient" adversaries A, $|\Pr[A \text{ outputs } b] - ½| \leq$ "negligible."

Discussion: CPA security

Pros

- A CPA-secure enc scheme is secure even if attacker can see many encryptions on messages of her choosing
- A very useful definition of security

In theory: "polynomially many"
In practice: ~2^{50} bytes

Cons

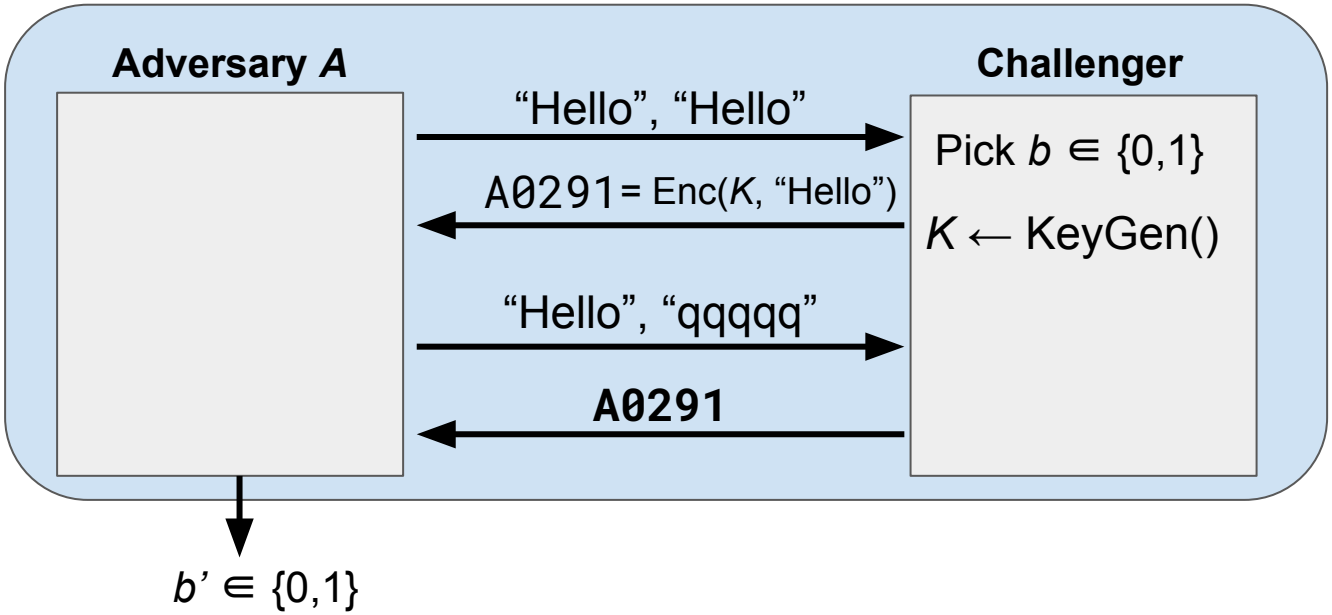
- CPA security does not require enc scheme to hide message **lengths**
  - In the CPA-security game: M_0 and M_1 must be the same length
  - To break CPA security, adversary must learn something other than message length
  - Hiding message lengths would be neat but is usually too costly.
- CPA security does not guarantee authenticity
- CPA security does not guarantee anything if attacker can somehow get *decryptions* of ciphertexts of her choosing

} **Will fix next week**

Important: CPA-secure encryption schemes must be randomized.

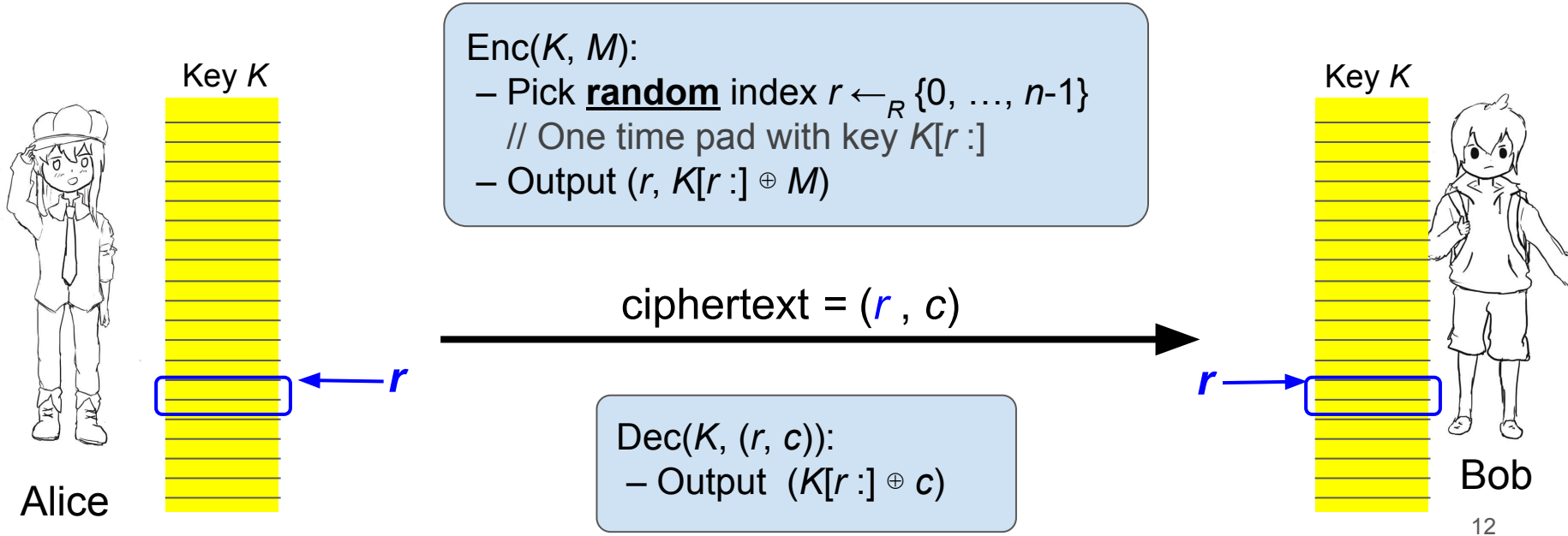
(*Or stateful)

If Enc() is deterministic, there is a direct attack on CPA security:



Warm up: An impractical CPA-secure encryption scheme

→ What if Alice and Bob could share a REALLY BIG random key? (e.g., $n = 2^{256}$ bytes)



Warm up: An impractical CPA-secure encryption scheme

Correctness: For every choice of encryption randomness r

$$\begin{aligned}\text{Dec}(K, \text{Enc}(K, M)) &:= \text{Dec}(K, (r, K[r:] \oplus M)) \\ &= K[r:] \oplus K[r:] \oplus M \\ &= M\end{aligned}$$

CPA Security. To give the intuition:

- It's almost certain that Alice will never use the same key bit twice
  - Probability of picking same r twice is ballpark: $\sim n^{-1/2} = (2^{256})^{-½} = 2^{-128}$ ("Birthday paradox")
- Then, by one-time pad security, ciphertext reveals no information about message
- Perfectly CPA secure!

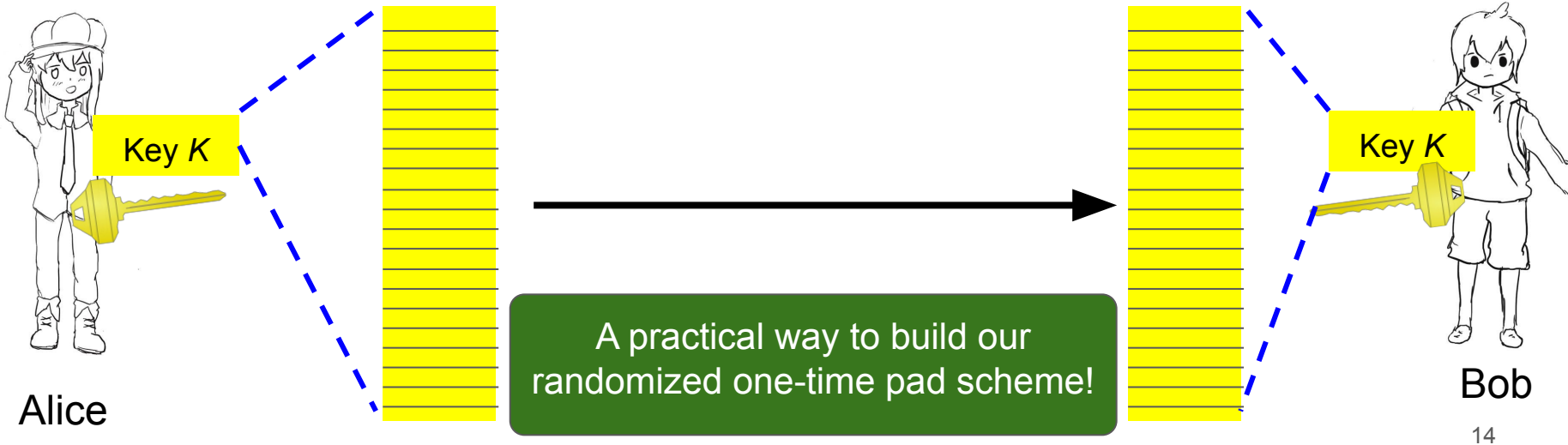
Efficiency? "Only" need 2^{256} bits of storage for key. (A billion 1TB disks = 2^{70} bytes)

Plan: Practical CPA-secure encryption scheme

– Alice and Bob share a short (e.g., 128-bit) **random** key *K*

– They can expand this key into a huge string of **as-good-as-random** bytes

→ Given key *K* can compute any bit of the huge string quickly

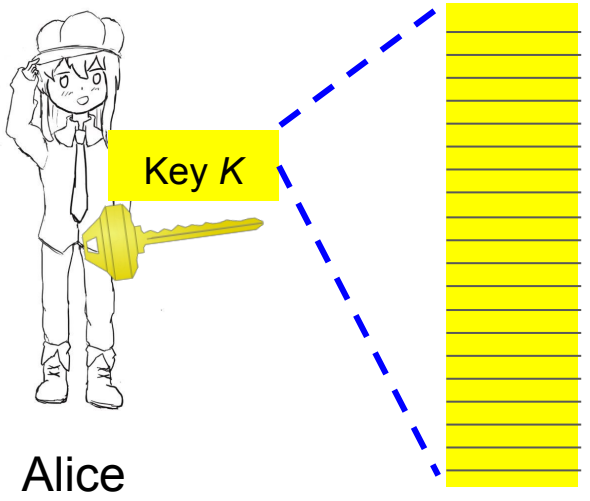


Plan: Practical CPA-secure encryption scheme

– Alice and Bob share a short (e.g., 128-bit) **random** key *K*

– They can expand this key into a huge string of **as-good-as-random** bytes

→ Given key *K* can compute any bit of the huge string quickly



Need a way to expand:
– a short random key into
– huge string of as-good-as-random bytes

Cryptography roadmap

	Secret-key crypto	Public-key crypto
Encryption	✅ One-time pad ✅ CPA security – Counter-mode encryption – CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	– Pseudorandom functions – Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem



Pseudorandom functions

Definition: Pseudorandom function (PRF)

Goal: Stretch short random key into many "as-good-as-random" bytes.

A pseudorandom function with:

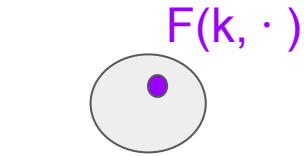
- key space **K**, // e.g., $\mathbf{K} = \{0,1\}^{256}$ 256-bit key
- input space **X**, and // e.g., $\mathbf{X} = \{0,1\}^{128}$ 128-bit input
- output space **Y** // e.g., $\mathbf{Y} = \{0,1\}^{128}$ 128-bit output

is an efficient deterministic algorithm $F : \mathbf{K} \times \mathbf{X} \rightarrow \mathbf{Y}$.

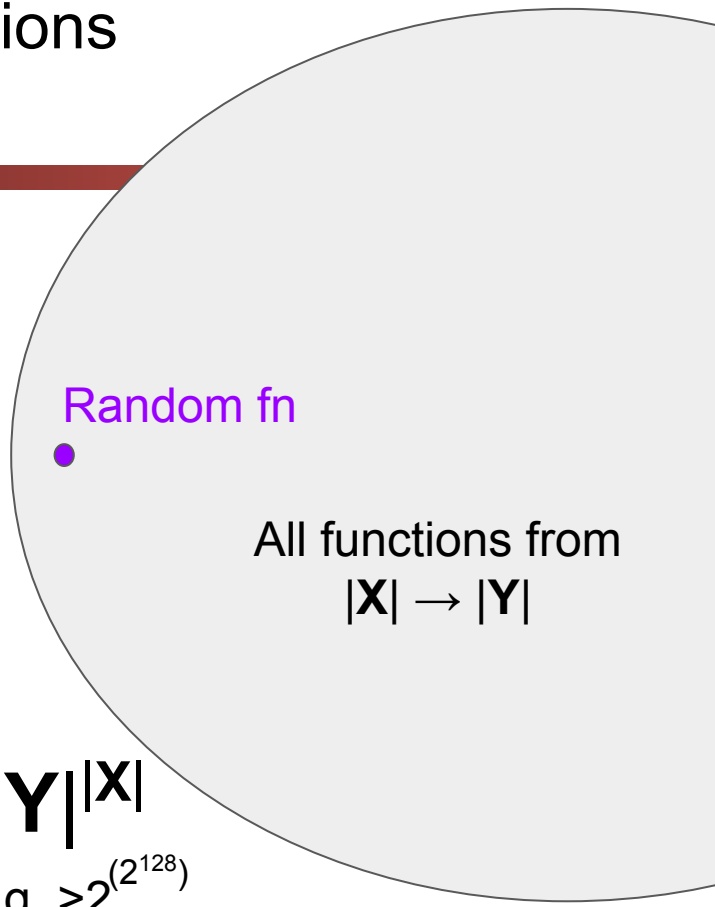
PRF Security:

- Informally: For a random key k, $F(k, \cdot)$ should "look like" a truly random function from $\mathbf{X} \rightarrow \mathbf{Y}$.

There are **many** more truly random functions than pseudorandom ones F(k, ·)

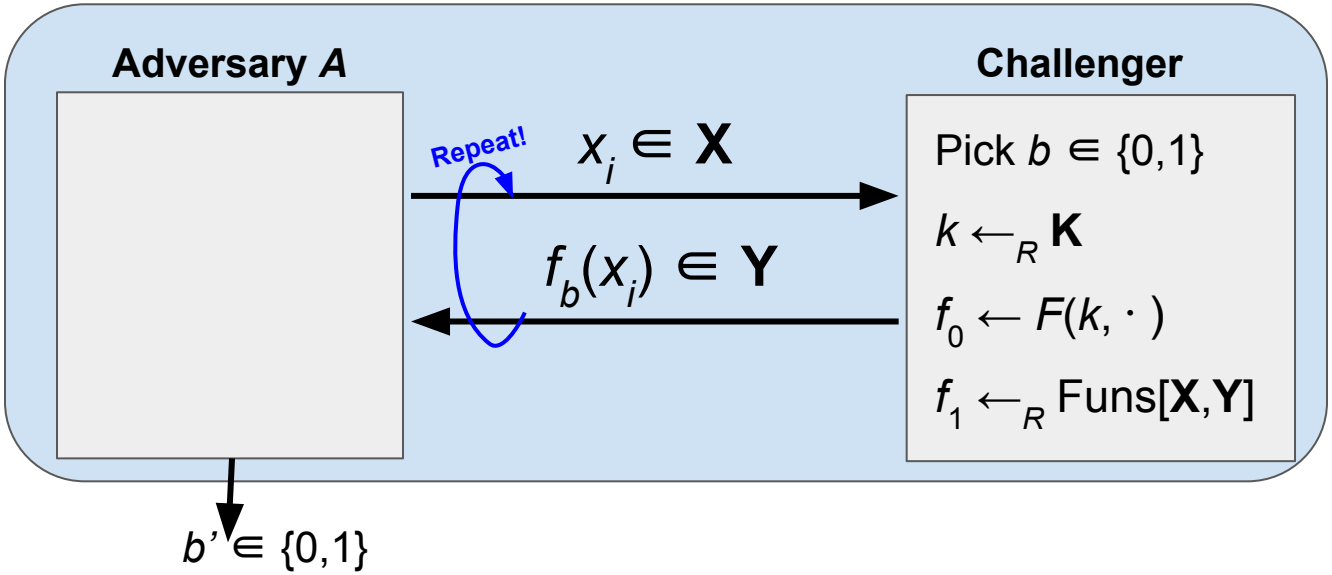


Keyspace |**K**|
e.g. 2^{256}



$|\mathbf{Y}|^{|\mathbf{X}|}$
e.g. $>2^{(2^{128})}$

PRF Security



A function F : **K** × **X** → **Y** is a *secure pseudorandom function* if,
for all "efficient" adversaries *A*, | Pr[*A* outputs *b*] - ½ | ≤ "negligible."

Example: Insecure PRF (from Dan Boneh)

Let **K** = **X** = **Y** = $\{0,1\}^{128}$

Define insecure PRF $F(k, x) := k \oplus x$

Notice: For any distinct *x* and *x'*, we have:

$$F(k, x) \oplus F(k, x') = (k \oplus x) \oplus (k \oplus x') = x \oplus x'$$

Attack in PRF game:

- Query challenger on x and x'. Get responses y and y'.
- If $x \oplus x' = y \oplus y'$, output "0" (PRF). Else output "1" (random).

If y and y' are random 128-bit strings, $\Pr[x \oplus x' = y \oplus y]$ is TINY 2^{-128}

$\Rightarrow$ Pr[adversary wins] = 1 - (small)

Constructing PRFs

- We do not know how to prove that PRFs exist
  - This would require proving P != NP and much more
- Instead, we **assume** that certain functions are PRFs
  - Assumptions are based on many many years of cryptanalysis
- Often build PRFs from block ciphers (in a moment)

- Example assumption:

  For the AES256 function, every algorithm running in time < 2^{80} wins the PRF security game with probability < ½ + 2^{-50}.
- Where do these magic constants 2^{80} and 2^{-50} come from?

Running time

- 2^{30} = ops/second on a laptop
- 2^{58} = ops/second on Fugaku supercomputer
- 2^{68} = crypto ops/second on Bitcoin network
- 2^{92} = crypto ops/year on Bitcoin network
- 2^{114} = crypto ops required to use enough energy to boil ocean
- 2^{128} = we design cryptosystems to defend against attacks using this much energy
- 2^{140} = crypto ops required to use one year of sun's energy

⇒ When key size is > 128 bits, brute-force attacks are infeasible
… other attacks may still be possible



Probabilities

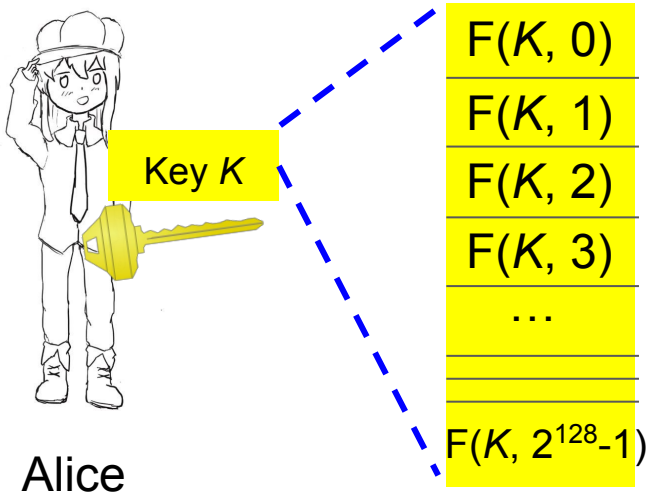
- 2^{-1} = probability that fair coin lands heads
- 2^{-19} = probability of getting struck by lightning next year
- 2^{-28} = probability of winning Mega Millions jackpot
- 2^{-56} = probability of winning 4x Mega jackpots in a row
- 2^{-64} = we often design cryptosystems to fail with this probability, at most

Plan: Practical CPA-secure encryption scheme

– Alice and Bob share a short (e.g., 128-bit) **random** key *K*

– They can expand this key into a huge string of **as-good-as-random** bytes

→ Using a PRF!



Cryptography roadmap

	Secret-key crypto	Public-key crypto
Encryption	✅ One-time pad ✅ CPA security – Counter-mode encryption – CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions – Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem



Block ciphers
(pseudorandom permutations)

Definition: Block cipher (a.k.a. pseudorandom permutation)

A block cipher with:

- key space **K** and // e.g., $\mathbf{K} = \{0,1\}^{256}$ 256-bit key
- input space **X** // e.g., $\mathbf{X} = \{0,1\}^{128}$ 128-bit "block size"

is a pair of efficient deterministic algorithms $E, D : \mathbf{K} \times \mathbf{X} \rightarrow \mathbf{X}$ ("encode," "decode")

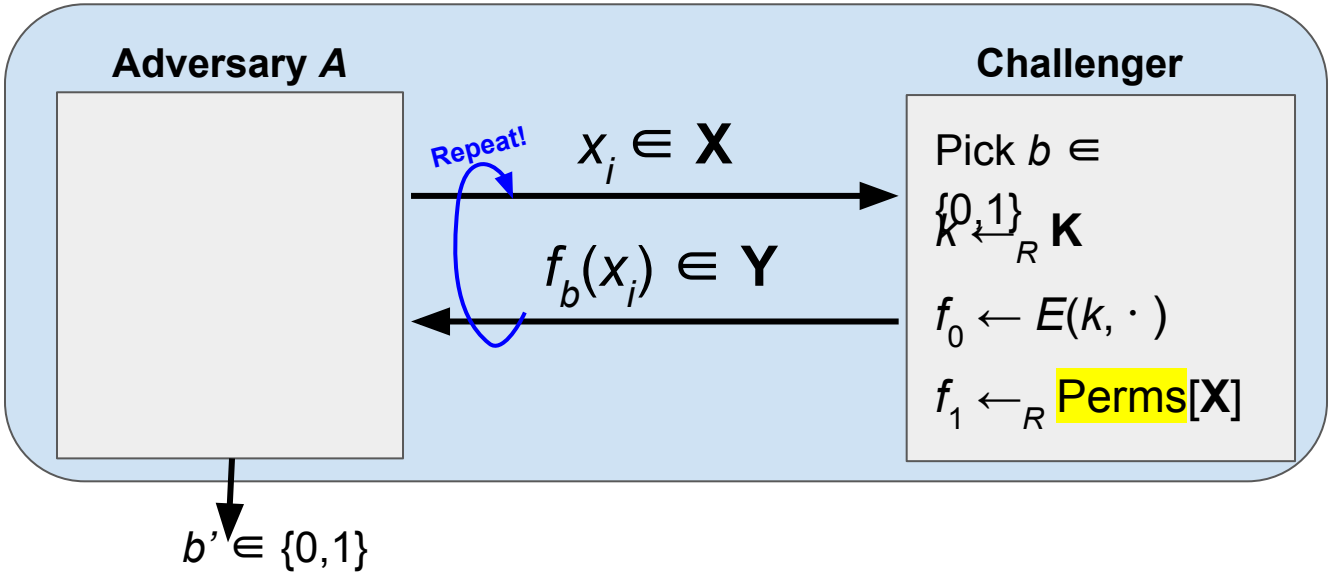
Required properties

1. **Distinct inputs map to distinct outputs under *E*:** (E(k,) is "permutation")
   For all keys k ∈ **K**, for all inputs x ≠ x' ∈ **X**: E(k, x) ≠ E(k, x')

2. ***D*(*k*, ·) computes the "inverse" of *E*(*k*, ·)**
   For all keys k ∈ **K**, for all inputs x ∈ **X**, D(k, E(k, x)) = x.

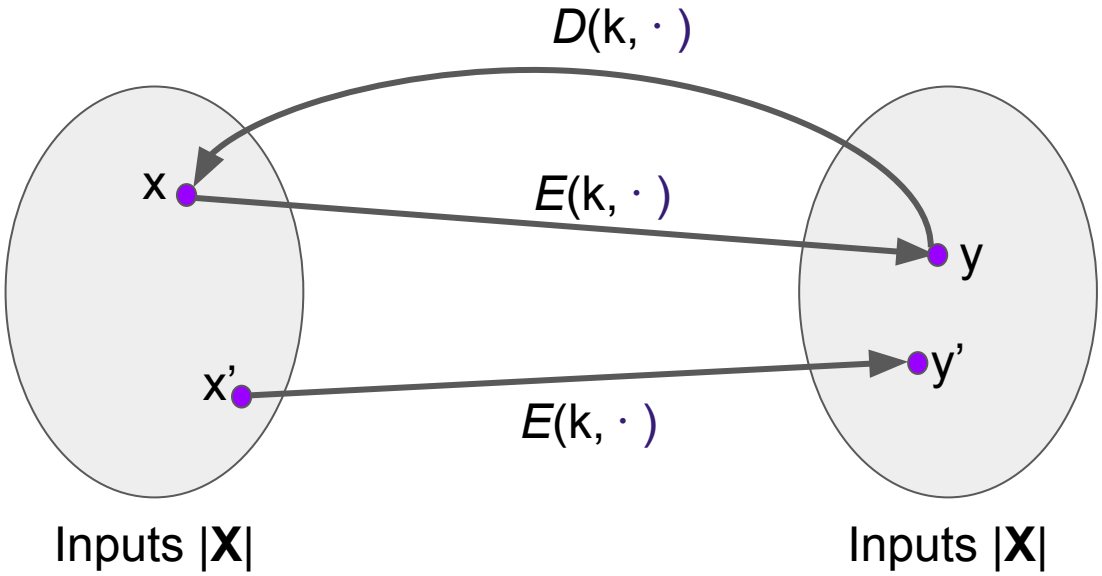
Block-cipher security:

Same as PRF security except $E(k, \cdot)$ should be indist from random permutation



A function F : $\mathbf{K} \times \mathbf{X} \rightarrow \mathbf{Y}$ is a *secure block cipher* (PRP) if,
for all "efficient" adversaries *A*, $|\Pr[A \text{ outputs } b] - ½| \leq$ "negligible."

Block cipher



Very important!

Water ≠ Coffee

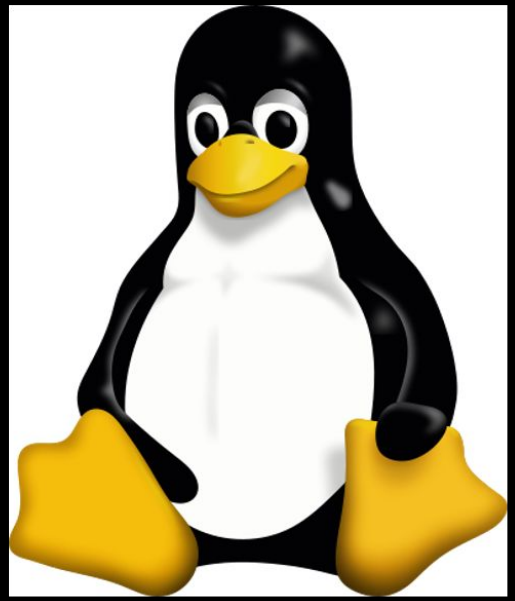
Seatbelt ≠ Airplane

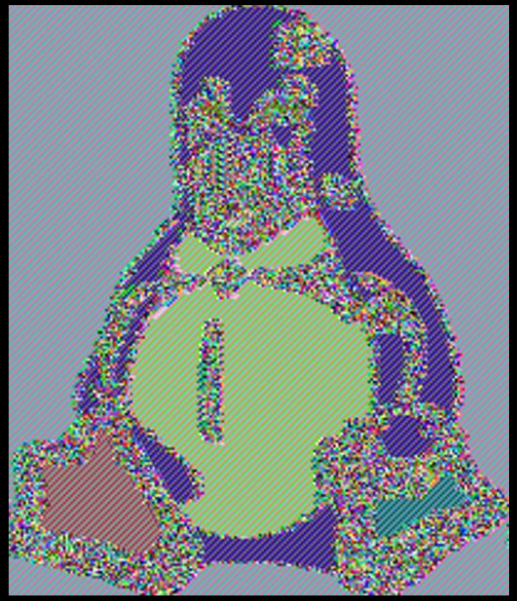
Block cipher ≠ Encryption scheme

Why is a block cipher not a CPA-secure encryption scheme?

- CPA-secure enc schemes are randomized
- Block cipher's *E*() function is deterministic
  - Encrypting same msg twice gives same ct! :-(
- Using block cipher directly to encrypt is called "ECB mode"
  - ECB = Electronic codebook mode

On Friday: How to use PRFs and block ciphers
(to build CPA-secure encryption schemes)







https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation

Using a block cipher as a PRF

Fact: If (E, D) is a secure block cipher with a large block size, e.g., $|\mathbf{X}| = 2^{128}$,
Then E is a secure pseudorandom function.

⇒ You can use a block cipher as a PRF.

- Historically, we built encryption schemes from block ciphers
  - NIST standardizes block ciphers
- In modern crypto, we often want PRFs
- Simple hack: Use block cipher as PRF

The most common block cipher today:

AES (Advanced Encryption Standard)

- Key size: 128, 192, or 256 bits ($\mathbf{K} = \{0,1\}^{128}$ or $\{0,1\}^{192}$ or $\{0,1\}^{256}$)
  - Most people: 128-bit keys
  - NSA requires: 256-bit keys for top-secret info
- Block size: 128 bits ($\mathbf{X} = \{0,1\}^{128}$)
  - Note: The block size is still always 128 bits, regardless of key size
- You need to know the details on this slide, but not the internals of AES
  - Here's a comic
- Common strategy: Repeat a simple "mixing" operation many times.
  - Challenge: Getting good performance

AES hardware acceleration

Modern CPUs have hardware instructions to accelerate AES

Reason 1: Speed. AES-based encryption runs at > 10 GB/s on my laptop

- Try yourself: `$ openssl speed -evp aes-128-gcm`
- The “GCM” stands for “Galois counter mode” (next lecture)

Reason 2: Security.

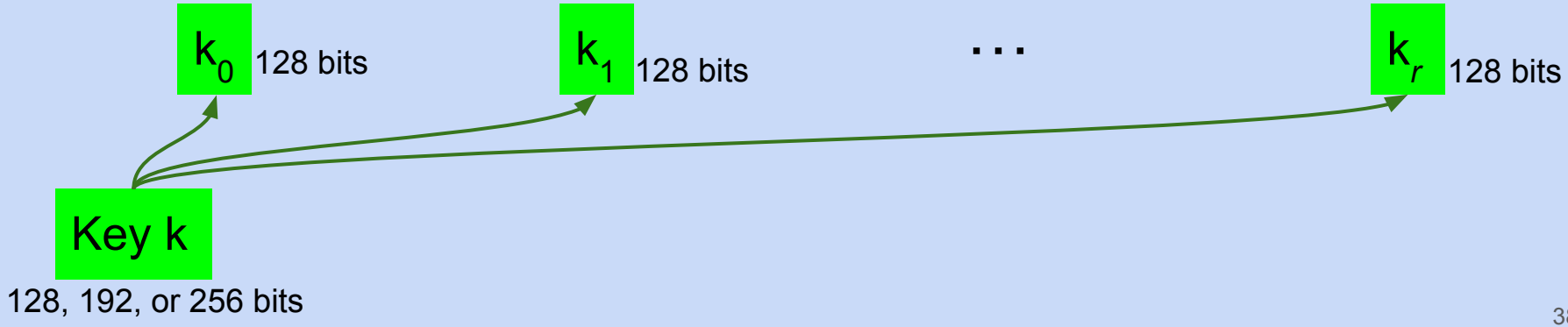
- It’s easy to mess up a software implementation of AES
- Running time of *E* depends on the bits of key ⇒ Sneaky side-channel attacks
- Hardware execution is “constant time” = independent of the key

AES History

- 1997–2000: NIST (National Institute of Standards and Technology) in the US held a competition to pick a new block cipher standard
  - David Wagner helped design one of the finalists, Twofish
- Out of the 5 finalists:
  - Rijndael, Twofish, and Serpent had really good performance
  - RC6 had okay performance
  - Mars had bad performance
- On any given computing platform, Rijndael was *never* the fastest
- But on every computing platform, Rijndael was *always* the second-fastest
  - Twofish and Serpent each had at least one compute platform they were bad at
- Rijndael was selected as the new block cipher standard → AES

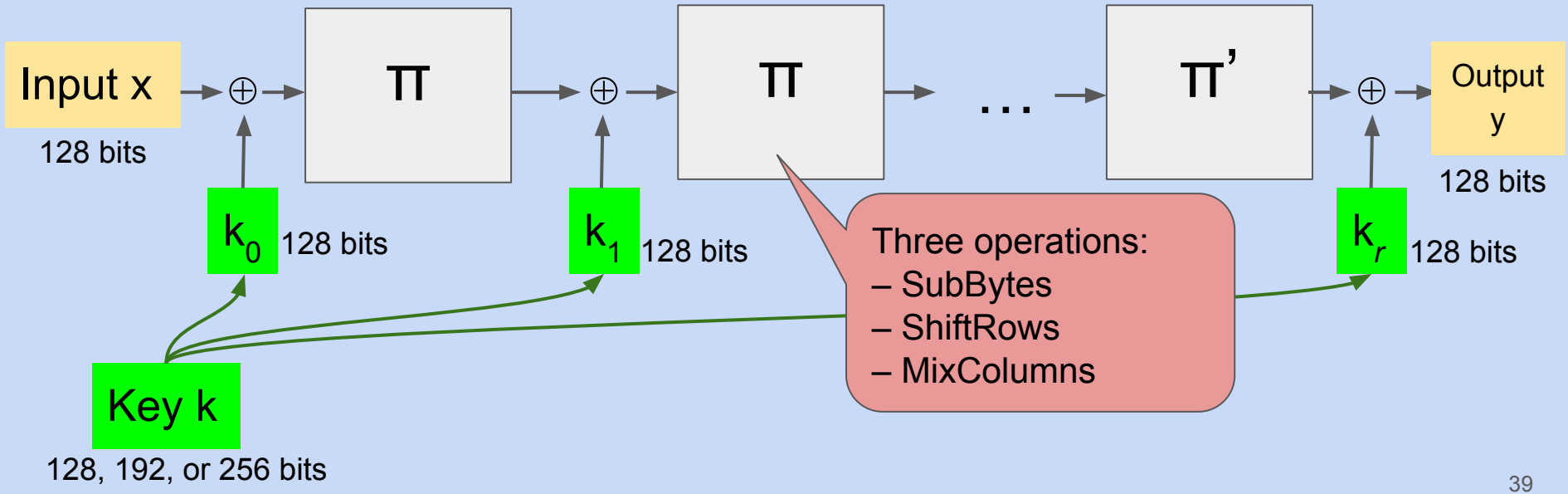
How AES implements *E*(k,)
Step 1: Expand key into many “round keys”

- Derive many round keys from the one block-cipher key – “key schedule”
  - 128-bit key → 10 rounds
  - 192-bit key → 12 rounds
  - 256-bit key → 14 rounds



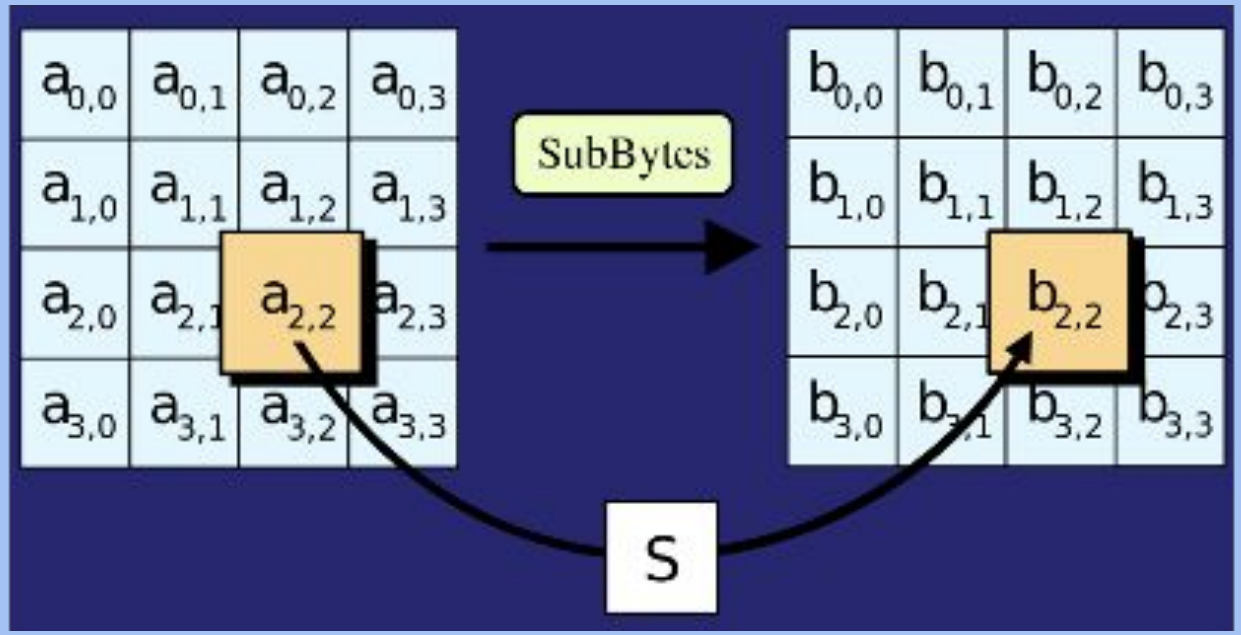
How AES implements *E*(k,)
Step 2: Repeatedly XORs round key with input, then mixes

Uses a simple invertible function $\pi : \{0,1\}^{128} \rightarrow \{0,1\}^{128}$



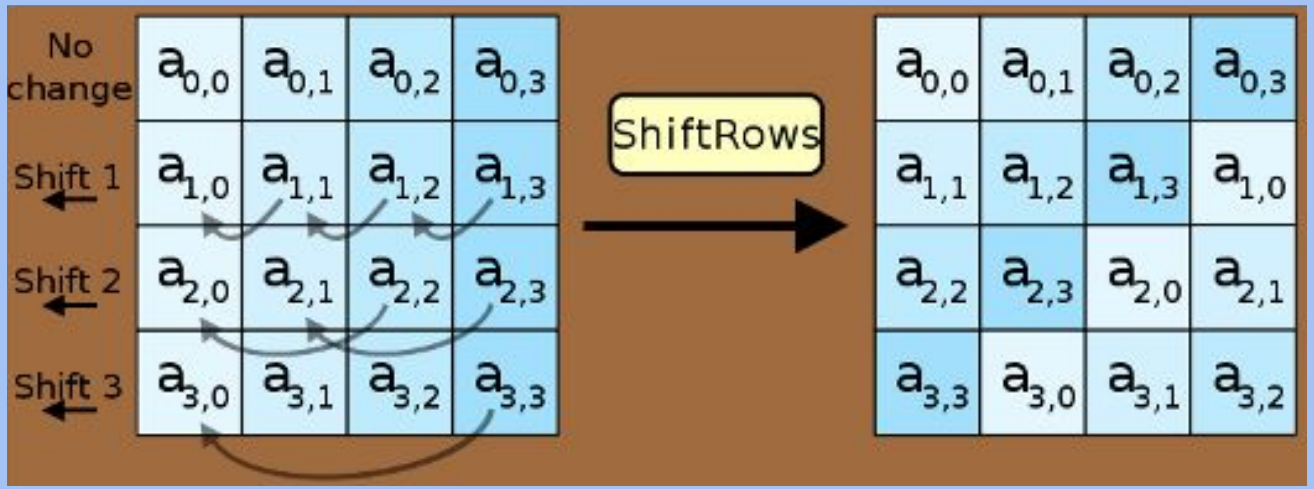
AES Algorithm: SubBytes()

- Treats 128-bit (16-byte) block input as a 4x4 matrix of bytes
- Replace each byte in the block with another byte using an 8-bit substitution box



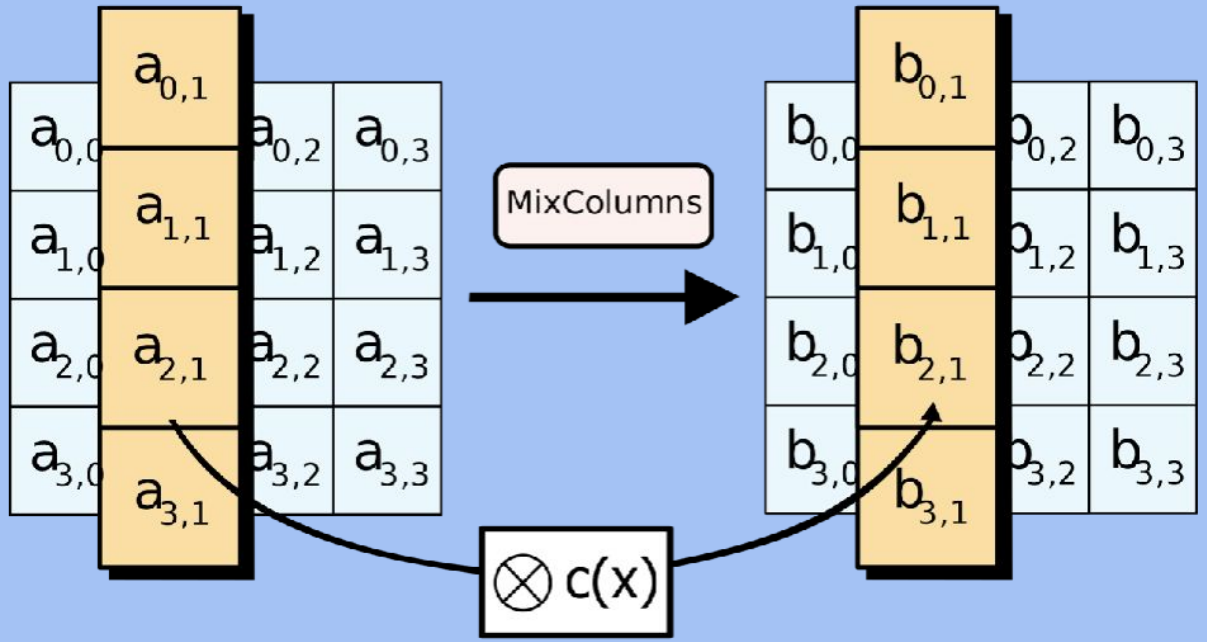
AES Algorithm: ShiftRows()

- Cyclically shifts the bytes in each row by a certain offset
- The number of places each byte is shifted differs for each row



AES Algorithm: MixColumns()

- Treats the 16-byte block as a 4 × 4 matrix and multiply it by by another matrix



AES (Advanced Encryption Standard)

- There is no proof that AES is a secure block cipher
  - We are very far from being able to prove such things
- We *assume* that it is secure, after 25+ years of cryptanalysis
  - The NSA uses AES-256 for secrets they want to keep secure for the 40 years
- **Takeaway**: AES is the modern standard block cipher algorithm
  - The standard key size (128 bits) is large enough to prevent brute-force attacks

DES (Data Encryption Standard)

- The first US-standardized block cipher
- Designed in late 1970s
- Block size = 64 bits, Key size = 56 bits
- NSA influenced two facets of its design
  - Altered some subtle internal workings in a mysterious way
  - Reduced key size from 64 bits to 56 bits
  - Made brute-force attacks feasible for government-scale attacker
- Key size is way too small – hours or days to brute-force
  - Small block size is also a big problem (e.g., Sweet32)
- Triple DES ("3DES") used in legacy applications until recently
  - Nests three layers of DES encryption

Take-away messages

- CPA security
  - How we define security for an encryption scheme
  - Allows adv to see encryptions of many messages of its choice
- Pseudorandom function
  - How to stretch 128 random bits into many random-looking bits
- Block cipher
  - Similar to a pseudorandom function, except with an inverse algorithm
  - Most important example: AES

Next time: How to use PRF/block cipher to build CPA-secure enc. scheme

Cryptography roadmap

	Secret-key crypto	Public-key crypto
Encryption	✅ One-time pad ✅ CPA security – Counter-mode encryption – CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem